DEPARTMENT OF
COMPUTER SCIENCE

CONTROL FLOW

DR JEREMIAH O. BANDELE

PhD Electrical and Electronic Engineering

University of Nottingham

LEARNING OBJECTIVES

At the end of the lesson, you should be able to:

- Illustrate branching in C.
- Illustrate iteration in C.
- Give examples of branching and iterative operations in C.

Branching

If we view each line of code in a computer program as a branch, then we can conclude that computer programs have many branches or lines of code.

Branching basically means that the computer can decide to move program control to another line of code instead of the usual movement from one line to the next line.

UNCONDITIONAL BRANCHING

With unconditional branching, some unconditional statements can move program control to either a specific statement or block without checking any condition.

Unconditional branching: goto

An example of an unconditional statement is the 'goto' statement.

You will notice that the 'goto' statement did not specify any condition before moving program control.

```
22    Get data inputs from user
23    goto print_data
24    calculate the data sum
25    print the data sum
26    print_data
```

CONDITIONAL BRANCHING

With conditional branching, the normal flow of operation in our code is also altered but where program control moves to is dependent on if some predetermined conditions are satisfied or not.

Conditional branching statements are also known as decision making or selection statements.

Conditional branching

There are three types of decision making statements. The three statements are the 'if', 'if-else' and 'if-else-if' statements. One point to take note of is that each of these statements are usually blocks consisting of one or more statements.

CONDITIONAL BRANCHING: IF

With the 'if' statement, program control will only move inside the 'if' block if some conditions are satisfied, or else, the entire block will be skipped.

```
if (boolean_expression) {
        /*  statement(s) will execute if the boolean expression
        is true */
}
```

Conditional branching: if

```
12    if (grade > 44) {
13        printf("The student passed");
14    }
```

CONDITIONAL BRANCHING: IF-ELSE

With the 'if-else' statement, if some conditions are satisfied, program control will move inside the 'if' block, otherwise, it will move to the else statement block.

```
if(boolean_expression) {
   /* statement(s) will execute if the boolean expression is true */
} else {
   /* statement(s) will execute if the boolean expression is false */
}
```

Conditional branching: if-else

```
16    if (grade > 44) {
17        printf("The student passed");
18    } else {
19        printf("The student failed");
20    }
```

CONDITIONAL BRANCHING: IF-ELSE-IF

With the 'if-else-if' statement, more that two choices are integrated into the decision making process.

```
if(boolean_expression 1) {
   /* Executes when the boolean expression 1 is true */
} else if( boolean_expression 2) {
   /* Executes when the boolean expression 2 is true */
} else if( boolean_expression 3) {
   /* Executes when the boolean expression 3 is true */
} else {
   /* executes when the none of the above condition is true */
}
```

Conditional branching: if-else-if

```
29    if (grade > 69) {
30        printf("The student scored an A");
31    } else if (grade > 44) {
32        printf("The student passed");
33    } else {
34        printf("The student failed");
35    }
```

CONDITIONAL BRANCHING: SWITCH

With the switch statement, we can execute one code block among many alternatives.

```
switch(expression) {

   case constant-expression  :
      statement(s);
      break; /* optional */

   case constant-expression  :
      statement(s);
      break; /* optional */

   /* you can have any number of case statements */
   default : /* Optional */
   statement(s);
}
```

Conditional branching: switch

```
 8    switch(grade) {
 9       case 70 :
10          printf("The student scored an A" );
11          break;
12       case 50 :
13          printf("The student passed" );
14          break;
15
16       default :
17          printf("The student failed" );
18    }
```

ITERATION/LOOPS

In general, statements are executed sequentially.

The first statement in a function is executed first, followed by the second, and so on.

However, sometimes a block of code needs to be executed several number of times.

LOOPS: WHILE

Here, statement(s) may be a single statement or a block of statements. The condition may be any expression, and true is any nonzero value. The loop iterates while the condition is true.

When the condition becomes false, the program control passes to the line immediately following the loop.

```
While (condition) {
        statement(s);
}
```

LOOP: WHILE

```
5       int num = 5;
6       while( num < 10 ) {
7           printf("value of num: %d\n", num);
8           num++;
9       }
```

Loops: do-while

A do-while loop works the same way as a while loop, except the fact that it is guaranteed to execute at least one time.

```
 9      do {
10          statement(s);
11      } while( condition );
```

LOOP: DO WHILE

```
5       int num = 5;
6       do {
7         printf("value of num: %d\n", num);
8         num++;
9       } while( num < 10 );
10  }
```

LOOPS: FOR

The init step is executed first, and only once.

Next, the condition is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute.

```
for ( init; condition; increment ) {
    statement(s);
}
```

LOOP: FOR

```
5    for( int num = 5; num < 10; num = num + 1 ){
6        printf("value of num: %d\n", num);
7    }
```

SUMMARY

- Branching means that the computer can decide to move program control to another line of code instead of the usual movement from one line to the next line.
- There are three types of decision making statements. The three statements are the 'if', 'if-else' and 'if-else-if' statements.
- Conditional branching statements are also known as decision making or selection statements.

FURTHER READING RESOURCES

- Kernighan, B. W., & Ritchie, D. M. (2002). The C programming language.
- Bailey, T. (2005). An introduction to the c programming language and software design. July-2005

Department of
Computer Science

Thank You